

Bedrock Model Multiplexer — Specification

Weighted Routing of Converse API Requests to Model Pools

Implementation-Agnostic Behavioral Specification

Overview

The **Bedrock Model Multiplexer** routes AWS Bedrock Converse API requests across a pool of models using weighted random selection, per-model circuit breakers, cross-model retry with failover, and real-time health monitoring. The system comprises five logical subsystems:

1. **Multiplexer** — orchestrator: owns model pools, delegates to the Request Handler, publishes events
2. **Request Handler** — per-request state machine: retry loop with cross-model failover
3. **Model Wrapper** — model adapter: stamps `modelId`, enforces timeout, records circuit breaker outcomes
4. **Circuit Breaker / Circuit Breaker Manager** — per-model three-state breaker (Closed → Open → Half-Open)
5. **Utilities** — weighted selection, error classification, health checks, validation, tracing, timers

Invariants:

- Every model pool has ≥ 1 primary (non-fallback) model.
- Model IDs are unique across all pools.
- A single Circuit Breaker Manager is the source of truth for all breaker state.
- The multiplexer never mutates the caller's input beyond stamping `modelId`.

Data Model

The multiplexer operates on the Bedrock Converse API request and response structures. The caller provides a complete Converse request *except* `modelId`, which the multiplexer stamps based on routing decisions.

Core Data Types

Type	Kind	Purpose
MultiplexerInput	Record	Converse API request with all fields except <code>modelId</code> (stamped by multiplexer)
MultiplexerConfig	Record	Top-level configuration: model list, timeout, max retries, tracing, circuit breaker settings
ModelConfiguration	Record	Per-model config: <code>modelId</code> (string), <code>weight</code> (number), <code>isFallback</code> (boolean)
MultiplexerStats	Record	Aggregate stats: success/rateLimit/failFast counts + latency metrics
ModelStats	Record	Per-model stats: counts + averageLatency
LatencyMetrics	Record	Percentiles: average, p50, p95, p99, min, max
ModelOutcome	Record	Result of one invocation: <code>modelId</code> , <code>outcomeType</code> , latency, timestamp
ErrorResponse	Record	Standardized error: code, message, details
EnhancedErrorResponse	Record	Extends <code>ErrorResponse</code> with: <code>errorType</code> , retryable flag, <code>retryAfterMs</code> , <code>recoverySuggestions</code>
SelectModelRequest	Record	Input to model selection: set of skipped model IDs
SelectModelResponse	Record	Output: selected <code>modelId</code> (nullable) + <code>isFallback</code> flag
ModelHealthStatus	Record	Per-model health: <code>circuitState</code> , <code>errorRate</code> , <code>requestsPerMinute</code>
SystemHealthStatus	Record	System-wide: healthy/degraded/unhealthy counts + aggregate metrics
ValidationResult	Record	<code>isValid</code> flag + list of validation errors
CircuitBreakerConfig	Record	<code>failureThreshold</code> , <code>recoveryTimeMs</code> , <code>successThreshold</code> , <code>failureWindowMs</code>
CircuitBreakerStatus	Record	<code>state</code> , <code>failureCount</code> , <code>successCount</code> , <code>lastOpenedAt</code> , <code>nextRetryAt</code>

Enumerations

Enum	Values
OutcomeType	SUCCESS, RATE_LIMIT, FAIL_FAST
CircuitBreakerState	CLOSED, OPEN, HALF_OPEN
ErrorType	VALIDATION, THROTTLING, MODEL_UNAVAILABLE, TIMEOUT, CANCELLED, CIRCUIT_OPEN, INTERNAL

Multiplexer (Orchestrator)

The Multiplexer is the top-level entry point. It owns two model pools (primary and fallback), a Circuit Breaker Manager, a Health Check Manager, and an optional Tracer. It publishes typed events via an observer/event-emitter pattern (see Event System section).

Operation	Inputs / Outputs	Semantics
<code>construct</code>	<code>config</code> $\rightarrow$ Multiplexer	Validate config, initialize subsystems, register models
<code>addModel</code>	<code>ModelConfiguration</code> $\rightarrow$ void	Create Model Wrapper with shared breaker, add to appropriate pool
<code>removeModel</code>	<code>modelId</code> $\rightarrow$ void	Destroy model, remove from pool/health/breaker
<code>processRequest</code>	<code>MultiplexerInput</code> $\rightarrow$ <code>ConverseResponse</code>	Create Request Handler, delegate, trace, emit events
<code>selectModel</code>	<code>SelectModelRequest</code> $\rightarrow$ <code>SelectModelResponse</code>	Primary pool first, then fallback; weighted random; skip open breakers
<code>getModel</code>	<code>modelId</code> $\rightarrow$ <code>ModelWrapper</code> or null	Lookup in primary then fallback pool
<code>reportOutcome</code>	<code>ModelOutcome</code> $\rightarrow$ void	Update stats, record latency, update health, emit events
<code>getStats</code>	$\rightarrow$ <code>MultiplexerStats</code>	Aggregate across all models + latency percentiles
<code>resetStats</code>	$\rightarrow$ void	Zero all counters, clear latency history
<code>getHealthStatus</code>	$\rightarrow$ <code>SystemHealthStatus</code>	Delegate to Health Check Manager
<code>isHealthy</code>	$\rightarrow$ boolean	True if <code>healthyCount</code> $>$ 0 and <code>unhealthyCount</code> $<$ $\max(1, \lfloor n/2 \rfloor)$
<code>destroy</code>	$\rightarrow$ void	Destroy all models, clear breakers, remove listeners

Delegate interface — The Request Handler communicates with the Multiplexer through a narrow interface of four operations: `selectModel`, `getModel`, `reportOutcome`, and `emitEvent`. This decouples the retry logic from the orchestrator.

Model selection algorithm (`selectFromPool`):

1. Filter out models in the `skippedModels` set (already tried for this request).
2. Filter out models where the circuit breaker disallows execution; publish `model-circuit-open-skipped` event.
3. Build weighted item list from remaining models.
4. Call `weightedRandomSelect` — returns null if list is empty.
5. Try the primary pool first; if null, try the fallback pool.

Latency percentiles — maintained via a ring buffer of the last 1 000 latencies, sorted on read:

$$p_k = \text{sorted}[\lfloor n \cdot k \rfloor], \quad k \in \{0.50, 0.95, 0.99\}$$

Request Handler — per-request state machine

Each call to `processRequest` creates a fresh Request Handler instance. It communicates with the Multiplexer via the delegate interface described above.

Retry loop (`process`):

1. Set `retryCount` $\leftarrow$ 0, `skippedModels` $\leftarrow$ $\emptyset$.
2. **while** `retryCount` $\leq$ `maxRetries`:
 - a. Call `selectModel(skippedModels)`. If null $\rightarrow$ raise error with code `NO_MODELS_AVAILABLE`.
 - b. Call `getModel(modelId)`. If null $\rightarrow$ add to `skipped`, increment `retry`, **continue**.
 - c. Call `model.invoke(input)`. On success $\rightarrow$ report outcome, return response.
 - d. On `RATELIMIT` error $\rightarrow$ report outcome, add model to `skipped`, increment `retry`, **continue**.
 - e. On `CIRCUITOPEN` error $\rightarrow$ report outcome, add model to `skipped`, increment `retry`, **continue**.
 - f. On `FAILFAST` error $\rightarrow$ raise immediately (no retry).
 - g. On unknown error $\rightarrow$ re-raise the original error as-is (no wrapping).
3. If loop exhausts $\rightarrow$ raise error with code `RETRIES_EXHAUSTED`.

Model Wrapper — Bedrock model adapter

Wraps a Bedrock runtime client for a single model. Owns: client instance, model configuration, circuit breaker reference, timeout setting.

Operation	Inputs / Outputs	Semantics
<code>construct</code>	<code>config</code> , <code>client?</code> , <code>timeoutMs?</code> , <code>breaker?</code> , <code>clientConfig?</code>	Create Bedrock runtime client with <code>clientConfig</code> passthrough
<code>invoke</code>	<code>input</code> , <code>abortSignal?</code> → { <code>response</code> , <code>outcome</code> }	Stamp <code>modelId</code> , check breaker, execute with timeout
<code>invokeStream</code>	<code>input</code> , <code>abortSignal?</code> → streaming response	Streaming variant via Converse Stream API; returns raw SDK response
<code>destroy</code>	→ void	Reset circuit breaker

Invoke flow:

1. `circuitBreaker.canExecute()` — if false, raise `CIRCUITOPENERROR` with outcome.
2. Construct a Converse API call with {`...input`, `modelId`}.
3. `executeWithTimeout(operation, timeoutMs, abortSignal)`:
 - Race: operation vs. timeout timer vs. abort signal.
 - First to resolve or reject wins; others are cleaned up.
4. On success → `breaker.recordSuccess()`, return `SUCCESS` outcome.
5. On abort/cancellation → return `FAILFAST` outcome (no breaker recording).
6. On timeout → `breaker.recordFailure()`, return `FAILFAST` outcome.
7. On throttling error → `breaker.recordFailure()`, raise `RATELIMITERROR` with outcome.
8. On other error → `breaker.recordFailure()`, raise `FAILFASTERROR` with outcome.

Streaming — `invokeStream` returns the raw streaming response from the Bedrock Converse Stream API. No custom iteration, chunk transformation, or token usage extraction is performed by the multiplexer; the caller consumes the stream directly using their platform’s streaming primitives.

Circuit Breaker — resilience pattern

Each model gets a Circuit Breaker instance managed by a single Circuit Breaker Manager. The breaker implements a three-state finite state machine.

	Parameter	Default	Meaning
Default configuration:	<code>failureThreshold</code>	5	Recent failures within sliding window to trip OPEN
	<code>recoveryTimeMs</code>	30 000	Time in OPEN before transitioning to HALF_OPEN
	<code>successThreshold</code>	2	Successes in HALF_OPEN to close the circuit
	<code>failureWindowMs</code>	60 000	Sliding window for counting recent failures

State transitions:

From	Event	To	Action
CLOSED	<code>recentFailures ≥ threshold</code>	OPEN	Record <code>lastOpenedAt</code> , reset <code>successCount</code>
CLOSED	<code>success</code>	CLOSED	Clear failure history
OPEN	<code>t − lastOpenedAt ≥ recoveryTimeMs</code>	HALFOPEN	Reset <code>successCount</code> (checked lazily on state query)
HALFOPEN	<code>success (count ≥ successThreshold)</code>	CLOSED	Clear failures, reset counts
HALFOPEN	<code>any failure</code>	OPEN	Record new <code>lastOpenedAt</code>

Circuit Breaker Manager — collection-level operations:

- `getBreaker(modelId)` — get-or-create with default config.
- `canExecute(modelId)` — returns true if breaker allows requests (or breaker doesn’t exist yet).
- `getAllStatus()` — snapshot of all breaker states.
- `removeBreaker(modelId)`, `clear()`, `resetAll()` — lifecycle management.

Failure window cleanup: on every `recordFailure` and `getRecentFailureCount`, failures older than `failureWindowMs` are pruned from the list.

Error Classification — error taxonomy

All AWS SDK errors are classified into two layers: a coarse `OutcomeType` for the retry loop, and a fine `ErrorType` for diagnostics.

Operation	Inputs / Outputs	Semantics
classifyError	error $\rightarrow$ OutcomeType	Throttling errors $\rightarrow$ RATELIMIT; thing else $\rightarrow$ FAILFAST
classifyErrorType	error $\rightarrow$ ErrorType	Fine-grained 10-variant classification error name/message
isThrottlingError	errorName $\rightarrow$ boolean	Matches 6 throttling error names (slow)
toErrorResponse	error $\rightarrow$ ErrorResponse	Normalize to {code, message, details}
createEnhancedError	error, errorType?, modelId? $\rightarrow$ EnhancedErrorResponse	Full diagnostic error with recovery suggestions
getRecoverySuggestions	ErrorType $\rightarrow$ list of strings	Human-readable recovery advice per type
getErrorMessage	error $\rightarrow$ string	User-friendly message by AWS error

Throttling error names (any of these $\rightarrow$ RATELIMIT):

ThrottlingException	TooManyRequestsException	ServiceQuotaExceededException
LimitExceededException	RequestLimitExceeded	RateLimitExceeded

Failover semantics: the multiplexer does *not* perform timed retries or backoff. On a recoverable error (RATELIMIT or CIRCUITOPEN), the failing model is skipped and a different model is selected immediately with zero delay. This is cross-model failover, not same-model retry.

Health Monitoring

The Health Check Manager tracks per-model metrics (request timestamps, response times, success/failure counts) in a sliding window (default 60s, max 1000 entries per model).

Model health: a model is *healthy* iff circuit state $\neq$ OPEN AND errorRate < 0.5 .

System health:

- *Healthy model:* isHealthy AND circuit = CLOSED.
- *Degraded model:* isHealthy AND circuit $\neq$ CLOSED (e.g. HALFOPEN).
- *Unhealthy model:* not isHealthy.
- System is healthy iff unhealthyCount $< \max(1, \lfloor \text{totalModels}/2 \rfloor)$ AND healthyCount > 0 .
- The $\max(1, \dots)$ ensures a single healthy model reports healthy: without it, $\lfloor 1/2 \rfloor = 0$ makes the condition $0 < 0$ (always false).

Health endpoint — a formatter for external consumers should provide:

- `getSimpleHealth()` $\rightarrow$ {status: “healthy” or “unhealthy”, code: 200 or 503} — suitable for load balancers.
- `getDetailedHealth()` $\rightarrow$ full system snapshot with per-model detail and aggregate metrics (totalRequests, successRate, averageLatencyMs, p99LatencyMs).

Weighted Selection — model routing

Operation	Inputs / Outputs	Semantics
weightedRandomSelect	list of (item, weight) $\rightarrow$ item or null	Draw one item proportional to weight
createWeightedItem	item, weight $\rightarrow$ (item, weight)	Constructor helper

Algorithm: given items with weights $w_1, \dots, w_n$, total $W = \sum w_i$:

$$r \sim \mathcal{U}(0, W); \quad \text{select item } k \text{ where } \sum_{i=1}^{k-1} w_i < r \leq \sum_{i=1}^k w_i$$

Returns null if the list is empty or $W \leq 0$.

Observability

Tracing

The Tracer provides optional distributed tracing (e.g. AWS X-Ray). If the tracing backend is unavailable, all operations silently no-op.

Operation	Inputs / Outputs	Semantics
<code>isEnabled</code>	$\rightarrow$ boolean	True iff tracing is configured and the back-end is loaded
<code>putAnnotation</code>	key, value $\rightarrow$ void	Add indexed annotation to current trace segment
<code>putMetadata</code>	key, value $\rightarrow$ void	Add metadata (only if <code>captureBodies</code> is enabled)
<code>traceOperation</code>	name, operation, metadata? $\rightarrow$ result	Create subsegment, execute, annotate outcome/latency, close
<code>generateRequestId</code>	$\rightarrow$ string	Unique request identifier (e.g. <code>req_{timestamp}_{random}</code>)

Tracing configuration: `enabled` (boolean), `serviceName` (default “bedrock-multiplexer”), `captureBodies` (boolean), `captureModelSelection` (boolean). When `captureModelSelection` is true, model selection gets its own subsegment with skipped/available model counts.

Event System

The Multiplexer publishes typed events via an observer pattern. Consumers subscribe to events for monitoring, logging, or integration with external systems.

Event	Payload
<code>request</code>	(input)
<code>success</code>	(null, outcome: <code>ModelOutcome</code>)
<code>error</code>	(error: <code>ErrorResponse</code> or null, outcome: <code>ModelOutcome</code>)
<code>model-circuit-open-skipped</code>	(<code>modelId</code>)
<code>model-added / model-removed</code>	(<code>modelId</code>)
<code>stats</code>	(stats: <code>MultiplexerStats</code>)
<code>stats-reset</code>	()
<code>model-selected</code>	(<code>modelId</code> , <code>isFallback</code> , <code>skippedCount</code>)
<code>model-invocation-start</code>	(<code>modelId</code> , <code>requestId</code>)
<code>model-invocation-complete</code>	(<code>modelId</code> , <code>requestId</code> , <code>latencyMs</code>)

Implementation note: ensure that unhandled `error` events do not cause the process to crash. Register a default no-op handler if your platform’s event system raises on unhandled error events.

Input Validation

Three validation scopes, each producing a `ValidationResult` (isValid flag + list of errors):

Scope	Key Rules
Model config	<code>modelId</code> required (non-empty string), <code>weight</code> $\in [0, 10000]$, <code>isFallback</code> required
Multiplexer config	<code>models</code> ≥ 1 , unique IDs, ≥ 1 primary, <code>timeout</code> $\in [1000, 300000]$ ms, <code>retries</code> $\in [0, 10]$
Circuit breaker config	<code>failureThreshold</code> $\in [1, 100]$, <code>recoveryTimeMs</code> $\in [1000, 300000]$, <code>successThreshold</code> $\in [1, 20]$, <code>failureWin</code>

Helper operations: `formatValidationErrors(result)` $\rightarrow$ human-readable string; `assertValid(result)` raises on failure.

Timer — timeout primitive

A cancellable timeout abstraction with operations: `start(ms)`, `cancel()`, `isRunning()`. Calling `start` while running cancels the previous timer first. Implementations should use platform-appropriate timer primitives.

Request Lifecycle

End-to-end pseudocode for a single `processRequest` call:

```
// 1. Caller invokes multiplexer
response = multiplexer.processRequest(input)

// Internal flow:
// 2. Multiplexer creates a new RequestHandler(input, delegate, maxRetries)
// 3. RequestHandler.process() enters retry loop:
//   a. delegate.selectModel({skippedModels})
//       → selectFromPool(primaryModels, skipped) // weighted random, skip open breakers
//       → if null: selectFromPool(fallbackModels, skipped)
//       → if null: raise NO_MODELS_AVAILABLE
//   b. delegate.getModel(modelId)
//   c. model.invoke(input)
//       → check circuitBreaker.canExecute()
//       → Converse API call with {...input, modelId}
//       → race: operation vs timeout vs abort signal
//       → on success: breaker.recordSuccess(), return {response, outcome}
//       → on error: breaker.recordFailure(), raise typed error with outcome
//   d. delegate.reportOutcome(outcome)
//       → updateModelStats(), recordLatency()
//       → healthCheckManager.recordSuccess/Failure()
//       → publish 'success' or 'error' event, publish 'stats' event
//   e. On RateLimit or CircuitOpen: skip model, retry with next model
//   f. On FailFast: raise to caller immediately
// 4. Response returned to caller (or error raised)
```